

Underwater Eye: An IoT Camera to Observe Underwater Creatures

Hanna Suzuki

Bedford High School
Bedford, MA, 01730, USA

Abstract

This work develops an Internet-of-Things (IoT) device, called Underwater Eye, to operate a waterproof camera module and observe underwater creatures at a close range. It is built with a Raspberry Pi computer, a motorized pulley, and a high-resolution, wide-angle camera sealed in a waterproof enclosure. It runs Python code that drops down the camera to the water, takes pictures and videos of underwater creatures, pulls the camera back up, and uploads the pictures and videos to the cloud. The pictures and videos can be shared/distributed via smartphone/tablet apps and Web browsers. Underwater Eye implements a terminal-based character interface and an iOS Siri-based voice command interface for the user to remotely control the camera.

X.1 Introduction

Over the summer, and even throughout the year, fish observation is a very popular activity. Whether it be at a pet store or an aquarium, observing fish can be fun and even meditating for some. Personally, I have been a pet owner of a total of 11 fish in my life, and I adore going to aquariums. There is something mesmerising about their environment and movement.

Frequently when I go to aquariums, the aquarium keepers will go into the tanks with body suits and feed the fish. Whenever I see this, I get jealous that I can't do something similar. I want to see the fish up close as well. This is what inspired me to create something that would make it possible for me.

Underwater Eye is a camera system to observe and record underwater creatures up close. It features a high-resolution, wide-angle camera sealed in a waterproof enclosure, a motorized pulley, and a low-footprint computer called Raspberry Pi. Through its character and voice user interfaces (CUI and VUI), Underwater Eye allows the user to remotely drop the camera down into a body of water (e.g., fish tank), take pictures and videos, pull the camera back up, and upload the contents to the cloud. The user can browse the contents via Web or mobile interfaces.

This chapter guides you to reproduce Underwater Eye by sharing how to assemble hardware components and how to set up software code. Potential future directions and ideas are also described, so you can extend/customize Underwater Eye in your own project.

X.2 Hardware Setup

Underwater Eye is built with a small (credit card-sized) computer called Raspberry Pi and its peripheral components. This section explains what components to prepare and how to assemble them.

X.2.1 Required Hardware Components

Underwater Eye requires the following hardware components.

- Raspberry Pi 3A+ (1x): This is used for the user to remotely control the underwater camera. Purchase a MicroSD card separately. It does not come with the Raspberry Pi. (The capacity of 16 or 32 GB is sufficient.) Amazon Standard Identification Number (ASIN): B07KKBCXLY. Adafruit Product ID: 4027.
- TT motor (a gearbox-attached motor) (1x): This is used to drop the camera into the water and pull it back up. It is a brushed DC motor with a gearbox attached. The gearbox reduces the motor's rotational speed to increase its torque (twisting force). It is useful in this project because the motor needs to drive a load that requires torque (i.e., camera) but does not need to spin very quickly. Its gear ratio is 1:90, which is twice as slow (and twice as powerful) as the standard 1:48 ratio. The motor can be powered with 3V to 6V. At 3V, 80 mA and 60 RPM without a load, and 500 mA when stalled. At 6V, 100 mA and 120 RPM without a load, and 1000 mA when stalled. Adafruit Product ID: 3802.
- Pimoroni Inventor HAT Mini (1x): This is a board that can be stacked on a Raspberry Pi's GPIO header. It has a high-current motor driver IC, called DRV8833, which can supply sufficient current to control a DC motor. This is required because a Raspberry Pi itself cannot power a DC motor sufficiently. 16 mA is the maximum current that can be drawn from each GPIO pin. (The total current over all 40 pins is limited to 50 mA.) DRV8833 can control two DC motors at 2.7 V to 10.8 V. It can supply 1,500 mA per motor continuously. Adafruit Product ID: 5736.
- Male-to-male jumper wires (2x): These are used to wire a TT motor to an Inventor HAT.
- Lego-compatible bracket for a TT motor (1x): This can be pushed onto the body of a TT motor, and it adapts the motor to Lego blocks. Adafruit Product ID: 3815.

- Lego-compatible cross axle for a TT motor (1x) and Lego-compatible bobbin (1x): These are attached to the TT motor to drop down and pull up the camera. ASIN: B0C4LFN1HD. The cross axle and bobbin can be replaced with a TT motor pulley (Adafruit Product ID: 3789).
- Raspberry Pi Camera Module 3 with 120-degree wide angle lens (1x): The standard 75-degree angle lens works fine, but choose 120 degrees if you prefer a wider angle of view. This camera can take 4608×2592 pixel pictures and 1080p50, 720p100 or 480p120 videos. Adafruit Product ID: 5658.
- 2-meter camera cable (1x): The standard cable is not long enough for this project. Two meters is the longest choice for a Raspberry Pi camera cable. Adafruit Product ID: 2144.
- Entaniya waterproof camera enclosure kit (1x): This enclosure is IP67 rated, which means it can withstand temporary submersion in water; for example, up to 1 meter deep for up to 30 minutes. It is compatible with Raspberry Pi Camera Modules 2 and 3. DigiKey Product ID: WC-01-SETB.
- Waterproof grease and calk. Any products on the market should work; for example, ASINs: B000DZFUNY and B0C9VYYCPB.

ASINs can be used for product searches at <https://www.amazon.com>, and Adafruit product IDs can be used at <https://www.adafruit.com>. Use <https://www.digikey.com> for DigiKey product IDs.

X.2.2 Sealing a Camera with a Waterproof Enclosure

First, replace the camera module's default cable with a long (2-meter) cable. Then, place the module on the base of a camera enclosure (Fig. 1).

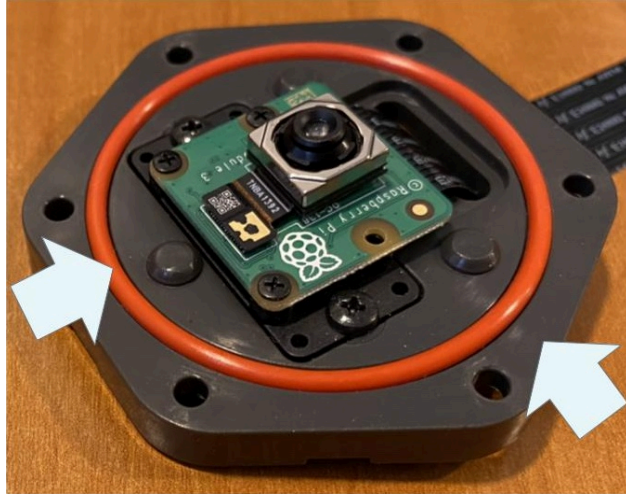


Fig. 1: Raspberry Pi Camera placed on an Enclosure Base

To make this enclosure watertight, apply waterproof grease onto the red ring and the surface of the enclosure base. (See white arrows in Fig. 1.) Also, apply waterproof calk to the outlet of the camera cable. (See a white arrow in Fig. 2.)

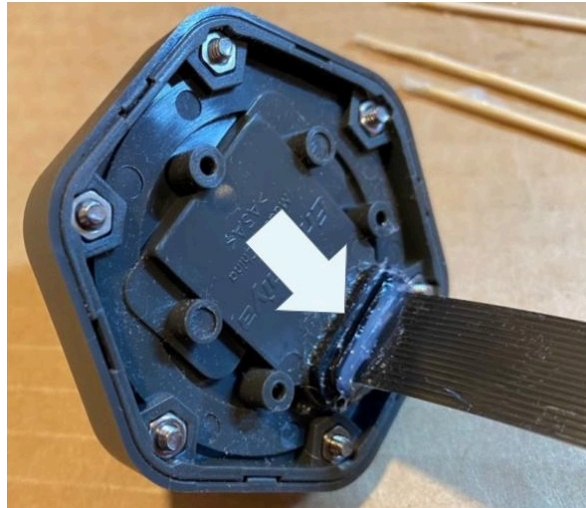


Fig. 2: Waterproof Calk applied to the Camera Cable Outlet

Attach a dome camera cover to the enclosure base with screws and connect the camera cable to the Raspberry Pi's cable connector (Fig. 3).

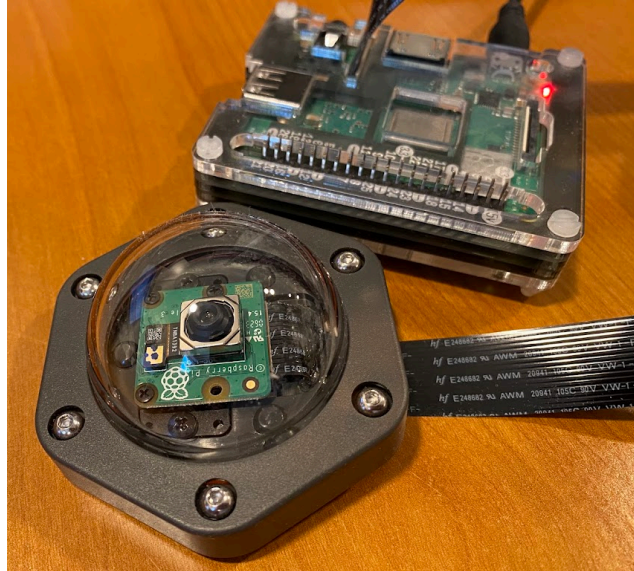


Fig. 3: Fully Assembled Camera Module

X.2.3 Connecting a TT Motor to an Inventor HAT Mini

Inventor HAT has wire connectors for DRV8833 on its backside. There are two connectors: “A” and “B.” This project uses the “A” connector, which consists of “+” (power supply) and “-” (ground) sockets. Using two male-to-male jumper wires, connect the “+” socket with the motor’s “+” terminal, and connect the “-” socket with the motor’s “-” terminal (Fig. 4). In Fig. 4, wires are not soldered to the motor’s terminals. They are hooked to the holes on the terminals and taped to the motor’s body.

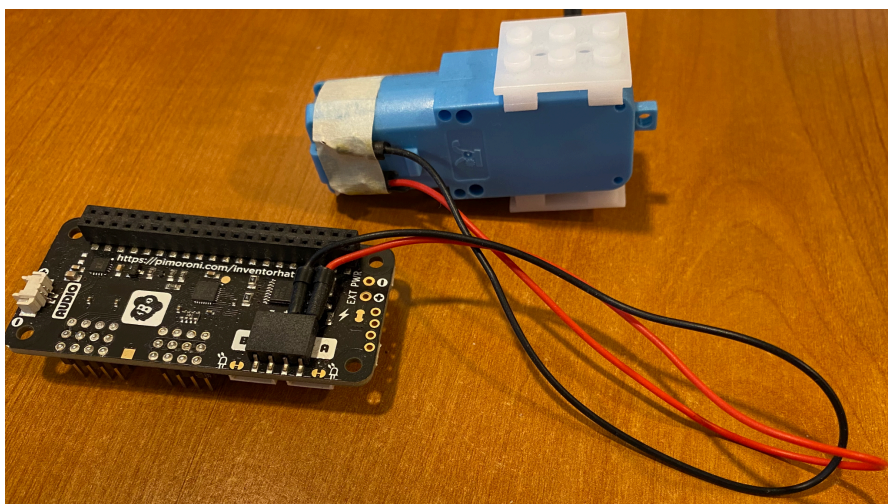


Fig. 4: A TT Motor (blue) connected to an Inventor HAT Mini (black)

Inventor HAT has 2x20 GPIO sockets; it can sit directly on top of Raspberry Pi's GPIO header (Fig. 5). In Fig. 5, the TT motor is secured to a Lego block base with a motor bracket (white). A cross axle is attached to the motor's gearbox shaft.

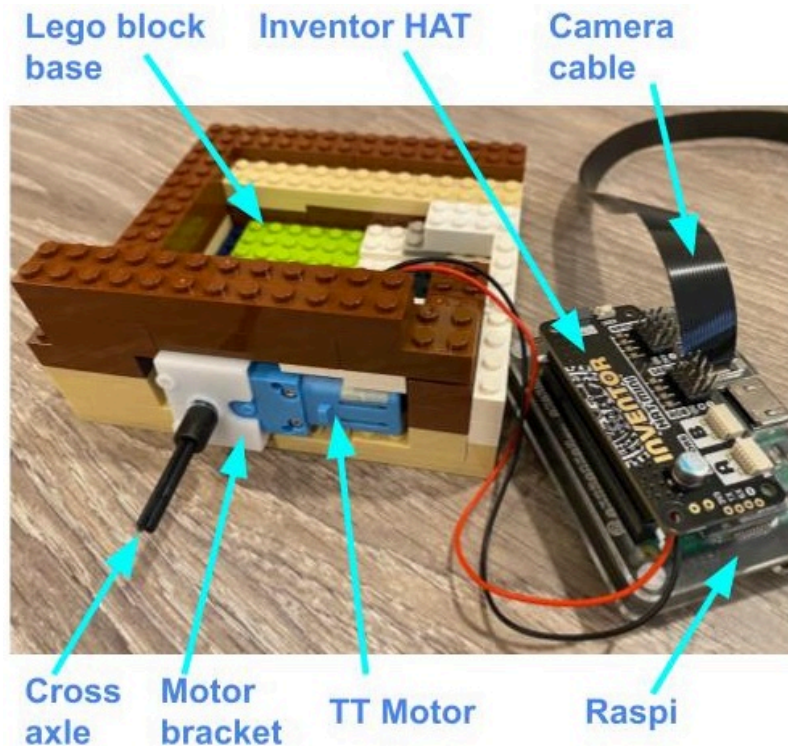


Fig. 5: Inventor HAT stacked on Raspberry Pi

X.2.4 Hanging a Camera on a TT Motor

Fig 6 shows how to hang the camera enclosure on the TT motor. Attach a metal mount to the camera enclosure, and tie a thin string like a fishing line to a hole on the mount. Attach a bobbin to the cross axle, and wrap the string around the bobbin. The camera enclosure's total weight is about 70 grams (2.5 oz), including the camera, metal mount and camera cable.

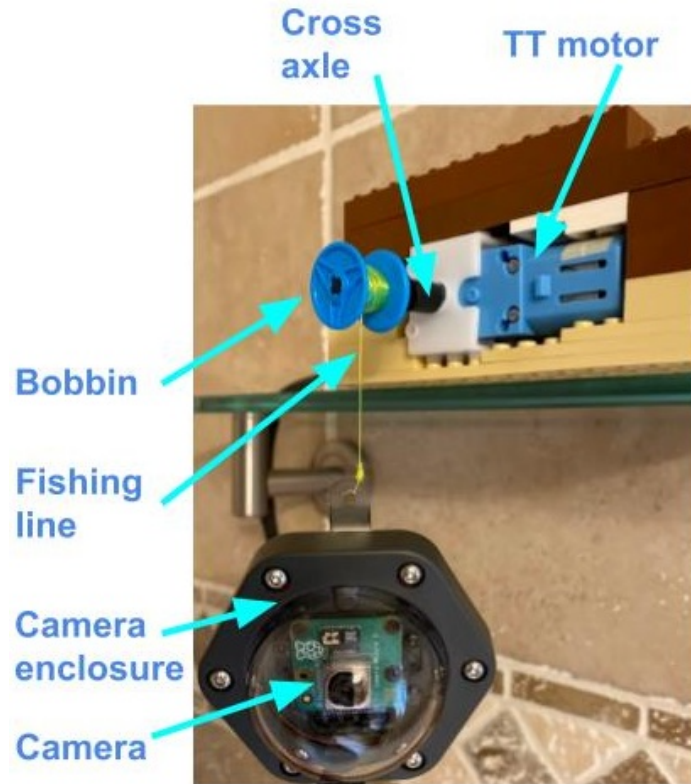


Fig. 6: Camera Enclosure Hanged on the TT Motor with a Fishing Line

X.3 Software Setup

Underwater Eye runs Python code in Raspberry Pi to receive commands from the user and control the underwater camera.

X.3.1 Setting up an Inventor HAT Mini

First, turn on your Raspberry Pi and update its operating system by running the following commands one by one on a Terminal.

- `sudo apt update -y`
- `sudo apt full-upgrade -y`

Then, enable I2C on your Raspberry Pi. Open the Raspberry Pi Configuration application from the Preferences menu and enable I2C in the Interfaces tab.

To install a Python library for the Inventor HAT Mini, run the following commands on a Terminal.

- `sudo pip3 install gpiod`
- `sudo pip3 install gpidevice`
- `sudo pip3 install inventorhatmini`

Run the following code to test the Inventor HAT Mini. This code spins the TT motor's shaft for two seconds, reverses its rotational direction, and spins the shaft in the reversed direction for two seconds.

```
import time
from inventorhatmini import InventorHATMini, MOTOR_A
from ioexpander.common import NORMAL_DIR, REVERSED_DIR

board = InventorHATMini(init_leds=False)
m = board.motors[MOTOR_A]
m.enable()

m.speed(0.5)
time.sleep(2)
m.stop()
time.sleep(1)

m.direction(REVERSED_DIR)
m.speed(0.5)
time.sleep(2)
m.stop()
```

This code is available in the `tests` folder at [1]. See [2] for the reference manual of Inventor HAT Mini.

X. 3.2 Python Code

Underwater Eye consists of three Python code, which are available in the `code` folder at [1]:

- `camera-up-down.py`: This code drops down or pulls up the camera for a specified time duration.
- `camera-pic.py`: This code takes a picture with the camera and uploads it to a cloud database called Kintone.
- `camera-vid.py`: This code takes a video with the camera for a specified time duration and uploads it to Kintone.

`camera-pic.py` and `camera-vid.py` use `iotutil.py` and `kintone.py`, which are also available at [1]. Make sure to place all these files in the same folder in your Raspberry Pi.

The first code, `camera-up-down.py`, is shown below. To pull the camera, run this code with a command like `python3 up 10`. This command pulls the camera for 10 seconds. To slowly drop down the camera, run this code with a command like `python3 down 10`. This command drops down the camera for 10 seconds.

```
import sys, time
from inventorhatmini import InventorHATMini, MOTOR_A
from ioexpander.common import NORMAL_DIR, REVERSED_DIR

board = InventorHATMini(init_leds=False)
motor = board.motors[MOTOR_A]
motor.enable()

def down(duration):
    motor.direction(REVERSED_DIR)
    motor.speed(0.95)
    time.sleep(duration)
    motor.stop()

def up(duration):
    motor.direction(NORMAL_DIR)
    motor.speed(0.95)
    time.sleep(duration)
    motor.coast()
    motor.stop()

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("USAGE: python3 camera-up-down.py up <duration (sec)>")
        print("USAGE: python3 camera-up-down.py down <duration (sec)>")
    else:
        if sys.argv[1] == "up":
            up( float(sys.argv[2]) )
        elif sys.argv[1] == "down":
            down( float(sys.argv[2]) )
        else:
            print("USAGE: python3 camera-up-down.py up <duration (sec)>")
            print("USAGE: python3 camera-up-down.py down <duration (sec)>")
    motor.disable()
```

In this code, the variable `motor` contains an instance of the `Motor` class, which has various methods to control a motor. Its `enable()`, `disable()`, `speed()` and `stop()` methods enable, disable, run and stop a motor, respectively. The `speed()` method takes a parameter value that specifies the duration of running the motor in seconds.

The second code, `camera-pic.py`, is shown below.

```
import subprocess, sys, time, kintone, iotutils
from iotutils import getCurrentTimeStamp

sdomain = "SUB-DOMAIN-NAME"
appId   = "APP-ID-NUMBER"
token   = "APP-TOKEN"
```

```

def takePic():
    timeStamp = getCurrentTimeStamp()
    picFile = "pics/" + timeStamp + ".jpg"
    command = "libcamera-still -n -t 500 -o " + picFile

    status = subprocess.run(command, capture_output=True, shell=True)
    if status.returncode == 0:
        print(timeStamp + " Photo captured.")
    else:
        print("Failed to capture a picture")
        sys.exit()

    fileKey = kintone.uploadFile(subDomain=sdomain,
                                apiToken=token,
                                filePath=picFile)

    if fileKey is None:
        sys.exit()

    memo = "Picture taken by the Underwater Eye system"
    payload = {"app": appId,
               "record": {"photo": {"value": [{"fileKey": fileKey}] },
                       "memo": {"value": memo} }}
    recordId = kintone.uploadRecord(subDomain=sdomain,
                                    apiToken=token,
                                    record=payload)

    if recordId is None:
        sys.exit()

if __name__ == "__main__":
    takePic()

```

To run this code, you need to complete the three lines in blue as follows:

- Specify a Kintone subdomain name in `sdomain = "SUB-DOMAIN-NAME"`. Replace `SUB-DOMAIN-NAME` with your Kintone subdomain name.
- Specify a Kintone application ID in `appId = "APP-ID-NUMBER"`. Replace `APP-ID-NUMBER` with your Kintone application ID.
- Generate a Kintone API token and specify it in `token = "APP-TOKEN"`. Replace `APP-TOKEN` with your Kintone API token. It is a credential to call the Kintone API.

See [3] for more details about these parameter settings.

Fig. 7 shows an example picture captured by `camera-pic.py`. It shows robotic fishes in a bathtub. The code automatically uploads a captured picture to Kintone, so the user can browse it with a Web browser or Kintone's mobile apps (Fig. 8) [4, 5].



Fig. 7: An Example Picture captured by Underwater Eye

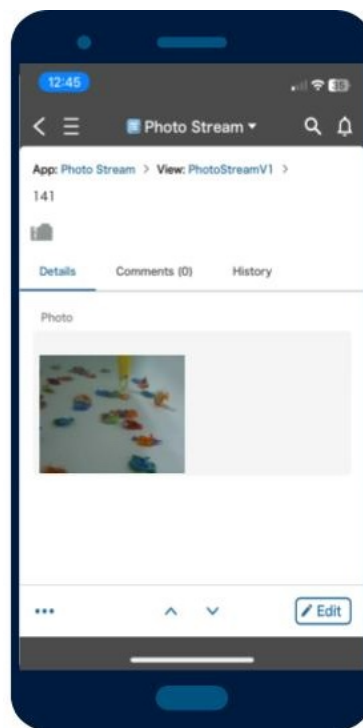


Fig. 8: An Example Picture stored in Kintone and browsed with Kintone's Mobile App

The third code, `camera-vid.py`, is shown below. To take a video, run this code with a command like `python3 camera-vid.py 5`, which takes a video for 5 seconds. Make sure to complete the three lines in blue as described earlier.

```
import subprocess, sys, time, kintone, iotutils
from iotutils import getCurrentTimeStamp

sdomain = "SUB-DOMAIN-NAME"
appId   = "APP-ID-NUMBER"
token   = "APP-TOKEN"

def takeVid(duration: float):
    timeStamp = getCurrentTimeStamp()
    vidFile = "videos/" + timeStamp + ".h264"

    command = "libcamera-vid -n -t " + str(int(float(duration) * 1000)) + " -o " + vidFile
    status = subprocess.run(command, capture_output=True, shell=True)
    if status.returncode == 0:
        print(timeStamp + " Video captured (" + str(duration) + " seconds).")
    else:
        print("Failed to capture a video")
        sys.exit()

    mp4File = "videos/" + timeStamp + ".mp4"
    command2 = "ffmpeg -y -i " + vidFile + " " + mp4File
    status = subprocess.run(command2, capture_output=True, shell=True)
    if status.returncode == 0:
        print(timeStamp + " MP4 file conversion completed.")
    else:
        print("Failed in MP4 file conversion.")
        sys.exit()

    fileKey = kintone.uploadFile(subDomain=sdomain,
                                apiToken=token,
                                filePath=mp4File)

    if fileKey is None:
        sys.exit()

    memo = "Video taken by the Underwater Eye system"
    payload = {"app": appId,
               "record": {"photo": {"value": [{"fileKey": fileKey}] },
                          "memo": {"value": memo} }}
    recordId = kintone.uploadRecord(subDomain=sdomain,
                                    apiToken=token,
                                    record=payload)

    if recordId is None:
        sys.exit()

if __name__ == "__main__":
    if len(sys.argv) != 2:
        takeVid(5)
    else:
        takeVid(float(sys.argv[1]))
```

X. 3.3 Setting up User Interfaces

Underwater Eye’s Python code (`camera-up-down.py`, `camera-pic.py` and `camera-vid.py`) can be executed from a remote machine via VNC (Virtual Network Computing) or SSH. Pi-compatible VNC clients are available on Windows, macOS, Linux, iOS, and Android [6, 7].

In addition to this character user interface (CUI), you can configure a voice user interface (VUI) with iOS/macOS Siri. This VUI allows you to give a voice command to Siri, which in turn executes Underwater Eye’s Python code on a remote Raspberry Pi via SSH. To set this up, open the Shortcuts app on iOS/macOS and create a new shortcut as shown in Fig. 9.

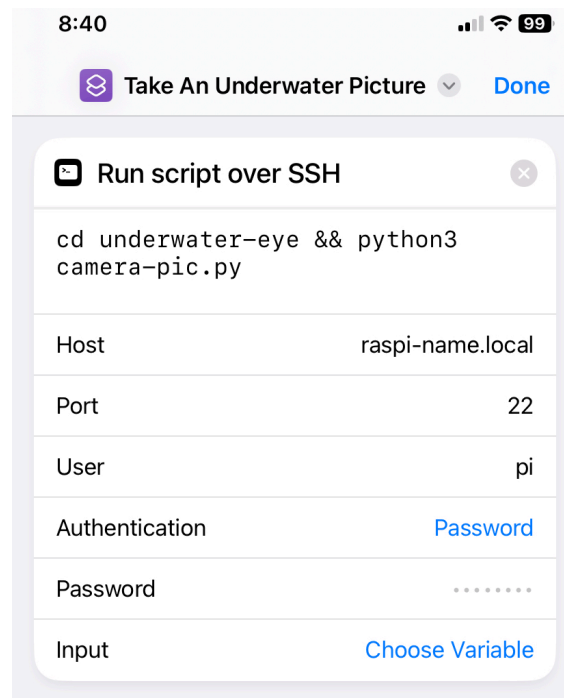


Fig. 9: An Example iOS Shortcut Configuration for Underwater Eye’s Voice User Interface

This example shortcut runs `camera-pic.py` on a remote Raspberry Pi via SSH. It assumes that `camera-pic.py` is placed at the `~/underwater-eye` folder on the Raspberry Pi. For the “Host” configuration, replace “raspi-name” with your Raspberry Pi’s name. Since this shortcut is named “Take an underwater picture,” you can say it to Siri, and then Siri will execute this shortcut. See [8] for the Shortcuts User Guide.

X. 4 Future Work

Underwater Eye can be extended in a few different ways. First, it can be enhanced with water quality sensors such as water temperature, pH, dissolved oxygen (DO), and total dissolved solids

(TDS) sensors. By dropping those sensors into the water in addition to a camera, Underwater Eye can sense the environmental conditions for underwater creatures rather than just observing and recording them. When it is used for a fish tank, water quality sensing can aid water change/replenishment planning and disease prevention.

Another direction is to build an automatic feeder and integrate it with Underwater Eye. By programming feeding time and frequency, the integrated system can observe and record underwater creatures at the feeding point.

A live streaming feature can be also added to Underwater Eye. In addition to taking pictures and videos, it would be nice and useful to observe underwater creatures live. For example, using YouTube Live [9], you can livestream the footage taken on the camera for a wider audience to see. You can use ffmpeg [10] or YouTube Live Streaming API [11] to integrate Underwater Eye with YouTube Live.

X. 5 Conclusion

Underwater Eye allows the user to remotely operate a waterproof camera module and observe underwater creatures at a close range. It is built with a Raspberry Pi, a motorized pulley, and a high-resolution, wide-angle camera sealed in a waterproof enclosure. Through its character and voice user interfaces (CUI and VUI), Underwater Eye allows the user to drop down the camera to the water, take pictures and videos of underwater creatures, pull the camera back up, and upload the pictures/videos to the cloud. The user can browse the pictures/videos via Web and mobile interfaces. Its Python code is available at [1].

References

- [1] <https://github.com/HSSBoston/underwater-eye>
- [2] <https://github.com/pimoroni/inventorhathmini-python>
- [3] <https://kintone.dev/>
- [4] <https://apps.apple.com/us/app/kintone/id674312865>
- [5] <https://play.google.com/store/apps/details?id=com.cybozu.kintone.mobile>
- [6] <https://www.realvnc.com/>
- [7] <https://tigervnc.org/>

[8] <https://support.apple.com/guide/shortcuts/>

[9] <https://www.youtube.com/live>

[10] <https://ffmpeg.org/>

[11] <https://developers.google.com/youtube/v3/live/getting-started>